

Technical Requirement Document

Here are the technical requirements based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data into Delta Tables

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a Delta Live Table pipeline to load customer and order data from CSV files into Delta tables.

Target Cluster Configuration:

- Cluster Name: Databricks Cluster for DLT
- Databricks Runtime Version: 10.4.x-scala2.12
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 Standard_DS3_v2
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: delta-lake, spark-csv

Source Data Details:

- Dataset Name: customerdata
- Location: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
- Format: CSV
- Description: Customer data in CSV format
- Dataset Name: orderdata
- Location: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata
- Format: CSV
- Description: Order data in CSV format

Target Data Details:

- Output Name: customer
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Customer data in Delta format
- Output Name: order
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard

- Format: Delta
- Description: Order data in Delta format with additional TotalAmount column

Job Flow / Pipeline Stages:

- Read customer CSV data into a Delta table named "customer"
- Read order CSV data into a Delta table named "order"
- Add a new column "TotalAmount" to the "order" Delta table

Data Transformations / Business Logic:

- Step: Read CSV data
- Description: Read customer and order CSV data into DataFrames
- Transformation Logic: Use spark.read.csv() to read CSV data
- Step: Add TotalAmount column
- Description: Add a new column "TotalAmount" to the "order" Delta table
- Transformation Logic: Use withColumn() to add a new column with the product of "PricePerUnit" and "Qty"

Error Handling and Logging:

- Log errors and exceptions to a logging table or file
- Implement retry mechanism for transient errors

TR-DLT-002: Clean and Process Customer and Order Data

Related Functional Requirement(s): FR-DLT-002

Objective: Implement data cleaning and processing logic to remove null and duplicate records from customer and order Delta tables.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: customer
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Customer data in Delta format
- Dataset Name: order
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Order data in Delta format

Target Data Details:

- Output Name: customer (cleaned)

- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Cleaned customer data in Delta format
- Output Name: order (cleaned)
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Cleaned order data in Delta format

Job Flow / Pipeline Stages:

- Remove rows with null values from the "customer" and "order" Delta tables
- Remove duplicate rows from the "customer" and "order" Delta tables

Data Transformations / Business Logic:

- Step: Remove null values
- Description: Remove rows with null values from the "customer" and "order" Delta tables
- Transformation Logic: Use dropna() to remove rows with null values
- Step: Remove duplicates
- Description: Remove duplicate rows from the "customer" and "order" Delta tables
- Transformation Logic: Use dropDuplicates() to remove duplicate rows

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-003: Create and Load ordersummary Table

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a Delta Live Table pipeline to create and load the "ordersummary" table with joined customer and order data using SCD type 2.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: customer
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Cleaned customer data in Delta format
- Dataset Name: order
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Cleaned order data in Delta format

Target Data Details:

- Output Name: ordersummary
- Location: Delta table in gen_ai_poc_databrickscoe.sdmc_wizard
- Format: Delta
- Description: Joined customer and order data in Delta format using SCD type 2

Job Flow / Pipeline Stages:

- Create the "ordersummary" table if it does not exist
- Join the "customer" and "order" Delta tables on the "CustId" column
- Load the joined data into the "ordersummary" table using SCD type 2

Data Transformations / Business Logic:

- Step: Join customer and order data
- Description: Join the "customer" and "order" Delta tables on the "CustId" column
- Transformation Logic: Use join() to join the DataFrames
- Step: Apply SCD type 2
- Description: Load the joined data into the "ordersummary" table using SCD type 2
- Transformation Logic: Use SCD type 2 logic to maintain history

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-004: Update ordersummary Table on Customer Changes

Related Functional Requirement(s): FR-DLT-004

Objective: Implement logic to update the "ordersummary" table when there are changes to the customer data.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: customer
- Location: Delta table in gen_ai_poc_databrickscoe.sdmc_wizard
- Format: Delta
- Description: Updated customer data in Delta format

Target Data Details:

- Output Name: ordersummary
- Location: Delta table in gen_ai_poc_databrickscoe.sdmc_wizard
- Format: Delta
- Description: Updated ordersummary data in Delta format

Job Flow / Pipeline Stages:

- Identify changes to the customer data

- Update the "ordersummary" table by making old records inactive and new records active
- Update the "StartDate" and "EndDate" columns accordingly to maintain history

Data Transformations / Business Logic:

- Step: Identify changes
- Description: Identify changes to the customer data
- Transformation Logic: Use join() and filter() to identify changes
- Step: Update ordersummary
- Description: Update the "ordersummary" table by making old records inactive and new records active
- Transformation Logic: Use SCD type 2 logic to update the "ordersummary" table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-005: Create and Load customeraggregatespend Table

Related Functional Requirement(s): FR-DLT-005

Objective: Implement a Delta Live Table pipeline to create and load the "customeraggregatespend" table with aggregated data from the "ordersummary" table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: ordersummary
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: ordersummary data in Delta format

Target Data Details:

- Output Name: customeraggregatespend
- Location: Delta table in gen_ai_poc_databrickscoe.sdlc_wizard
- Format: Delta
- Description: Aggregated customer spend data in Delta format

Job Flow / Pipeline Stages:

- Create the "customeraggregatespend" table if it does not exist
- Aggregate the "TotalAmount" column from the "ordersummary" table by "Name" and "Date"
- Load the aggregated data into the "customeraggregatespend" table

Data Transformations / Business Logic:

- Step: Aggregate data
- Description: Aggregate the "TotalAmount" column from the "ordersummary" table by "Name" and "Date"

- Transformation Logic: Use `groupBy()` and `agg()` to aggregate data

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-006: Implement Delta Live Tables

Related Functional Requirement(s): FR-DLT-006

Objective: Implement the above requirements using Delta Live Tables to ensure data is processed and loaded correctly.

Target Cluster Configuration: Same as TR-DLT-001

Job Flow / Pipeline Stages:

- Implement the above requirements using Delta Live Tables

Data Transformations / Business Logic: Same as above technical requirements

Error Handling and Logging: Same as TR-DLT-001